

ASYNC/AWAIT — GUIA DE ESTUDO

1. O que é programação assíncrona?

Em JavaScript, algumas operações podem levar um tempo para serem concluídas, como abrir uma página, fazer uma requisição, localizar um elemento ou realizar uma ação no navegador.

Essas operações são chamadas de **assíncronas** porque o JavaScript pode continuar executando outras instruções enquanto elas ainda estão sendo processadas.

Em testes automatizados, isso pode ser um problema, pois uma etapa pode tentar começar antes que a anterior tenha terminado.

Exemplo:

```
abrirPagina();
clicarBotao();
validarResultado();
```

Se `abrirPagina()` ainda não terminou, o teste pode tentar clicar no botão antes de ele estar disponível.

2. O que significa `async`?

A palavra **`async`** indica que uma função possui operações assíncronas.

```
async function realizarTeste() {
  // código do teste
}
```

Uma função marcada com `async` pode utilizar o `await` para aguardar operações assíncronas.

☐ **Para lembrar:**

`async` = esta função trabalha com operações assíncronas.

3. O que significa `await`?

A palavra **`await`** significa **aguardar**.

Ela faz com que o código espere a operação assíncrona terminar antes de continuar para a próxima instrução.

```
await page.goto("https://exemplo.com");
await page.click("#entrar");
```

A execução será:

Abrir página → esperar terminar → clicar → esperar terminar → continuar.

Normalmente, `await` é utilizado dentro de uma função `async`.

4. `async` + `await`

Os dois trabalham juntos:

```
async function login() {
  await page.goto("https://exemplo.com/login");
  await page.fill("#email", "teste@email.com");
  await page.fill("#senha", "123456");
  await page.click("#entrar");
}
```

Nesse exemplo, o Playwright executa as ações seguindo a sequência definida pelo teste.

Fluxo:

1. Abre a página;
2. Aguarda a operação;
3. Preenche o e-mail;
4. Preenche a senha;
5. Clica em entrar;
6. Continua o teste.

5. Por que isso é importante no Playwright?

O **Playwright** realiza diversas operações que são assíncronas, como:

- `page.goto()` → acessar uma página;
- `page.click()` → clicar em um elemento;
- `page.fill()` → preencher um campo;
- `page.locator()` → trabalhar com elementos;
- `expect()` → realizar verificações.

Por isso, é muito comum encontrar:

```
await page.goto(url);
await page.fill("#email", email);
await page.fill("#senha", senha);
await page.click("#entrar");
```

O `await` ajuda a manter o fluxo do teste organizado e previsível.

🔗 EXEMPLO DE TESTE DE LOGIN

```
test("realizar login", async ({ page }) => {

    await page.goto("/login");

    await page.fill("#email", "usuario@email.com");

    await page.fill("#senha", "123456");

    await page.click("#entrar");

    await expect(page).toHaveURL("/home");
});
```

Observe que o teste é declarado como:

```
async ({ page }) => {
```

e as operações do Playwright utilizam:

```
await
```

⚠ CUIDADO!

Não pense que `await` significa simplesmente “**parar o programa inteiro**”.

Ele aguarda a **Promise daquela operação assíncrona** dentro do fluxo da função.

A ideia principal é:

“**Não avance para esta próxima etapa enquanto esta operação não terminar.**”

☐ RESUMO PARA A PROVA

Conceito	Significado
Assíncrono	Operação que pode levar tempo para terminar
async	Define uma função como assíncrona
await	Aguarda uma operação assíncrona terminar
Promise	Representa o resultado futuro de uma operação assíncrona
Playwright	Utiliza operações assíncronas para controlar o navegador

★ FRASE PARA MEMORIZAR

`async` permite trabalhar com operações assíncronas e `await` aguarda uma operação terminar antes de continuar o fluxo do código.

🔑 Para o Playwright:

- `async` → define o teste como assíncrono
- `await` → aguarda cada ação do navegador

Resultado → testes mais organizados, previsíveis e fáceis de entender.